

UD7.2: Optimización. Índices

Índice

Introducción.....	2
Tipos de índices.....	2
Cuando Oracle NO usa el índice.....	2
Create index.....	3
Información sobre los índices.....	3
El optimizador.....	4
Modificaciones de una tabla.....	4
Degradaciones de un índice.....	5

Introducción

Un índice es una colección bien estructurada de una serie de valores que aparecen en una columna indexada de una tabla, permitiendo acceder directamente a esta lista de valores y aumentando la eficiencia temporal en los accesos a las filas. Un índice también puede estar compuesto por más de una columna e, incluso, de diferentes tablas. Para crear un índice sobre una tabla, este debe pertenecer al esquema del usuario que crea el índice, poseer el privilegio INDEX sobre el objeto o poseer el privilegio del sistema CREATE ANY INDEX. Si el objeto pertenece a otro esquema es obligatorio poseer este último privilegio.

Tipos de índices

La creación de un índice viene de la acción de optimizar los accesos de lectura de una tabla y la búsqueda de los valores de entrada. De este modo, existen diferentes tipos de índices atendiendo a los algoritmos y estructuras de datos que usan para localizar los datos de interés. Principalmente existen los siguientes tipos de índices:

- En árbol (B* Tree)
Son los más normales. **Buenos para claves primarias y columnas selectivas**
- De mapa de bits (Bitmap) **Buenos en columnas poco selectivas** (poco distintos)
- Particionados
- Basados en funciones **Buenos en resultados de funciones** o campos calculados

Los gestores avanzados resuelven las consultas así :

ANALIZADOR SINTÁCTICO	Comprueba si la sentencia es sintacticamente válida
--------------------------	---

ANALIZADOR SEMÁNTICO	Comprueba si las referencias a los objetos son válidos.
-------------------------	---

FILTRO DE SEGURIDAD	Comprueba si el usuario tiene permisos
------------------------	--

PLANIFICADOR	Determina el plan de ejecución para la sentencia SQL.
--------------	---

Hai dos formas de obtener los datos por parte del gestor:

- Escáner completo (full scan)

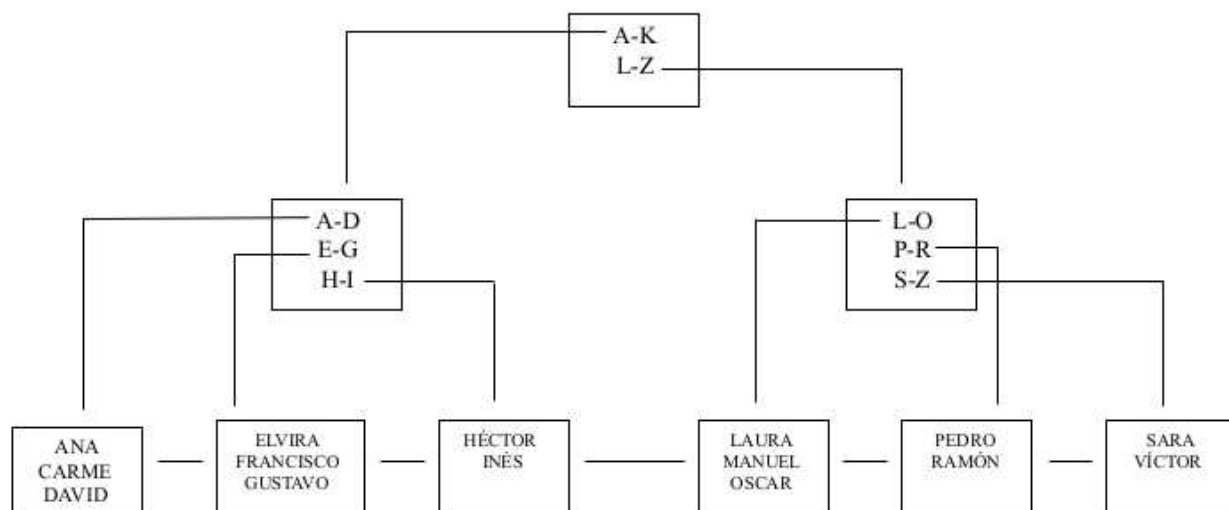
Es la forma más fácil para el gestor de obtener los datos. Todas las fila son leídas y llevadas a memoria. El gestor lee todos los bloques de disco comenzando por el primero al último.

- Uso de índices

Por este método, el gestor es capaz de traer los datos de disco sin tener que leer toda la tabla.

Supongamos que realizamos la sentencia *select dni from clientes*. Claro está que queremos seleccionar todas las tuplas de la tabla clientes por lo que el planificador hará un full scan. Si la sentencia es *select dni from clientes where nombre='Carmen'* el planificador, si decide hacer un full scan deberá leer posiblemente miles de tuplas para alomejor solo devolver unas cuantas. Los índices pueden ayudarnos a mejorar el tiempo de respuesta de un escáner completo muy costoso.

Esto puede hacerse [añadiendo a un índice todas las columnas referenciadas por la query: select, las columnas del join, del where y del order by.](#)



Coste de los índices

Mantener una estructura de índices consume espacio en disco, CPU y I/O. Debemos tener cuidado con el uso de los índices pues ralentizan los inserts por un factor de 3. Es decir, un insert en una tabla que tenga tres índices será 10 veces más lento que el insert original. Debemos tener esto en cuenta con aplicaciones donde el insert sea muy intensivo (no son las habituales)

Cuando Oracle NO usa el índice

Cuando se crea un índice sobre una columna se debe tener en cuenta, al usar el índice en una cláusula WHERE, que existen casos en los que no se usa ese índice.

Estos casos son los siguientes:

- Cuando la columna se usa en una expresión: en este caso se usa un índice de función.
- Cuando se usa el operador LIKE y no se conoce el comienzo de la cadena.
- Cuando se usa el operador <> o !=
- Cuando se usa el valor NULL, ya que los valores nulos no están en el índice.

Los valores nulos, logicamente no son indexados por lo que consultas tipo IS NULL y similares harán un full scan. Para que use el índice hai que usar un índice concatenado entre el campo y uno non nulo o un valor literal.

- Cuando se usa NOT IN, ya que estos valores tampoco están en el índice.

Cuando se crea un índice con más de una columna se debe tener en cuenta que el orden de estas importa. Un índice se utiliza cuando en la condición de WHERE de una operación de consulta se encuentra la primera columna del índice.

Cuando creamos un índice con varios atributos estamos creando un índice concatenado. Supongamos la siguiente consulta

select dni from clientes where nome='Ana' and apellido='López'.

Podemos tener cuatro escenarios:

- No existe índice : Full Scan
- Hay índice en nombre y no en apellido
- Tenemos dos índices, uno en nombre y otro en apellido : hará un INDEX MERGE y se verá un AND EQUALS en el plan
- Tenemos un índice concatenado

La diferencia en tiempo entre la primera y segunda opción suele ser del 50%, entre la primera y la tercera del 30%, y entre la primera y la cuarta del 10%. En estos casos deberíamos tener un índice concatenado.

Buenas prácticas

Estos son unos consejos en cuanto al uso de índices:

- No crees índices en tablas pequeñas ya que nunca van a ser usados.
- Crea un índice si normalmente se hacen consultas que muestren menos del 15% de la tabla, si no, Oracle hará un full scan, que lógicamente le costará menos.
- Realiza índices en las condiciones de xuncións de los join y en todas las columnas que aparecen en el Where.
- Cuanto más "único" sea un atributo, mejor funcionará el índice.
- Cuanto más largo sea el rango de valores de un atributo mejor funcionará el índice B* Tree
- El orden de los atributos importa a la hora de crear el índice. Coloca los atributos más usados al principio de la lista. Si creamos un índice en at1,at2,at3 las consultas dependientes de col1, col1-col2, col1-col2-col3,... usarán el índice mientras que las otras no.
- Tener índices tiene una sobrecarga en las operaciones de borrado, inserción y actualización. No recargues las tablas de índices y borra los que no sean usados.
- Coloca los índices en un tablespace separado (por ejemplo INDEX). Esto reducirá las colisiones de disco al acceder a los datos y índices al mismo tiempo.
- Crea índices con funciones (ej. create index idx_nombre on clientes(upper(nombre));) según la consulta.

Create index

Para crear un índice se usa el comando CREATE INDEX con una sintaxis bastante amplia, pero que se puede resumir como sigue:

```
CREATE [UNIQUE | BITMAP | MULTIVALUE ] INDEX [esquema].nombreIndice ON  
[ cláusula de Cluster | cláusula de Tabla | cláusula de Bitmap]  
[ USABLE | UNUSABLE | {DEFERRED | IMMEDIATE} INVALIDATION ];
```

La cláusula de tabla tiene la siguiente sintaxis:

```
[esquema].nombreTabla [aliasTabla] ({columna expresiónColumna [ASC | DESC]})
```

se indican las columnas que intervienen en el índice separadas por comas y con el valor ASC o DESC si se quiere ordenar la columna indexada de forma ascendente o descendente, respectivamente. El siguiente comando crea un índice sobre el campo clave de la tabla productos:

```
CREATE INDEX ix_Producto ON Productos(codProducto);
```

El siguiente ejemplo crea un índice sobre las columnas nombre y apellidos de la tabla empleado:

```
CREATE INDEX ix_NomEmpleado ON Empleado(nombre, prApellido, sgApellido);
```

Los índices suelen crearse en un tablespace diferente a las tablas y, si es posible, incluso en un disco diferente. Para ello, se usa la opción **TABLESPACE**. Además, se puede usar la cláusula **STORAGE** con la misma función y sintaxis que se usa para las tablas.

Ejemplo

```
CREATE INDEX ix_NomEmpleado ON Empleado(nombre, prApellido, sgApellido)  
TABLESPACE indices;
```

Información sobre los índices

Son varias las vistas que se pueden usar para disponer de información sobre la explotación de los índices y analizar su posible deficiencia. Estas vistas son DBA_INDEXES y DBA_IND_COLUMNS. La primera contiene información del índice y la segunda de cada una de las columnas que forman un índice.

El optimizador

Los gestores, a la hora de resolver una consulta, deben optar por distintas maneras de acceder físicamente a los datos. En definitiva, deben elegir entre recurrir todas las tuplas de la tabla (full scan) o bien usar uno o más índices, y el orden en que se realizan. De hacer esto, se encarga una parte del gestor llamada planificador. Una vez que decide un camino, no lo evalúa cada vez que se realiza la consulta si no que guarda esa decisión y la aplica de cada vez.

Cada vez que se inserta o borra un excepcional número de tuplas, es posible que las estadísticas en las que se basó el planificador o optimizador para decidir el camino de una consulta ya no sean válidas. Así pues, es preciso de vez en cuando, obligarlo a que analice de nuevo las tablas y actualice sus estadísticas sobre ellas, haciendo esto, el planificador tendrá una información más actualizada sobre los datos que contienen las tablas que le permitirá planear mejor el desarrollo de las consultas. Esto redundará en una respuesta más eficiente (en tiempo) de la base de datos.

Para hacer esto, Oracle cuenta con una sentencia llamada **ANALYZE**

Para que el optimizador refresque las estadísticas sobre la tabla o el índice podemos usar el comando

ANALYZE TABLE | INDEX nombre [{COMPUTE | ESTIMATE } STATISTICS [SAMPLE n ROWS[PERCENT]]]

Ya que esta sentencia se puede ejecutar estando la base de datos operativa, es recomendable hacerlo por la noche o en períodos de inactividad para molestar el menos posible a los usuarios que estén accediendo a la base de datos. En esta versión de Oracle está planificada una tarea (job) que hace esto cada cierto tiempo sin que nosotros tengamos que preocuparnos por el asunto. Aunque hay gente que recomienda que después de insertar un montón de datos o borrarlos, analizar la tabla; da mejores resultados borrar el índice y recrearlo de nuevo.

Examinar todas las filas de una tabla con COMPUTE STATISTICS no es práctico para tablas muy grandes. Llevará mucho tiempo o fallará debido a la falta de espacio en el tablespace temporal. En estos casos, estimar las estadísticas usando un tamaño entre lo 10 y el 25% es suficiente.

Para analizar un índice podemos

ANALYZE INDEX nombre VALIDATE STRUCTURE;

Lo siguiente que podemos hacer es ver su análisis en index_stats

```
SELECT * FROM INDEX_STATS WHERE NAME='nombre';
```

Para saber si un índice se usa podemos monitorizarlo durante un tiempo con las sentencias:

```
ALTER INDEX index MONITORING USAGE;
```

```
ALTER INDEX index NOMONITORING USAGE;
```

Luego podemos consultar la columna USED en la vista

```
V$OBJECT_USAGE
```

Modificaciones de una tabla

Podemos ver si una tabla es altamente modificada o no, es decir, podemos ver el número de inserts, updates y deletes hechos en ella

```
COL Ins HEA 'Inserciones'
```

```
COL Act HEA 'Actualizaciones'
```

```
COL Bor HEA 'Borrados'
```

```
COL Fecha FOR A8 'Fecha'
```

```
COL Tabla FOR A15 HEA 'Tabla';
```

```
SELECT TABLE_NAME Tabla, INSERTS Ins, UPDATES Act, DELETES BOR,  
TIMESTAMP Fecha
```

```
FROM DBA_TAB_MODIFICATIONS WHERE TABLE_OWNER='TIENDA';
```

Degradaciones de un índice

Cuando una columna puede cambiar de valor y se ha creado un índice asociado a ella, el índice que contiene la lista de valores no sustituye el valor viejo por el nuevo, sino que se elimina y se añade en un nuevo bloque. Esto hace que un índice sufra una degradación suficiente que afecte al rendimiento. También puede ocurrir que, durante el uso de un índice, este no esté usando y ni vaya a usar todo el espacio asignado en la configuración inicial del almacenamiento. También se puede detectar que los bloques no se llenan en su totalidad, obteniéndose una tasa media de ocupación considerablemente baja. En estos casos se recomienda gestionar la reedición del índice.

Para detectar la amortización del espacio se puede usar la siguiente consulta, que pretende obtener los bloques usados y los asignados de un índice:

```
SELECT BR_BLK+LF_BLK "Bloques usados", BLOCKS "Bloques reservados"  
FROM INDEX_STATS WHERE NAME='nombre';
```

Una tasa puede ser baja cuando PCT_USED es inferior al 70 %, la profundidad del árbol es mayor que cinco o si la división de Del_lf_row/Lf_rows es superior a 0,15. Una consulta ejemplo podría ser la siguiente:

```
SELECT PCT_USED, HEIGH, ROUND(DEL_LF_ROW/LF_ROWS) FROM  
INDEX_STATS WHERE NAME='nombre';
```

La solución para reorganizar un índice puede ser modificarlo, o eliminarlo y volverlo a crear. La primera opción suele ser más eficiente que la segunda, aunque debe evaluarse el comportamiento de un índice alterado. El comando que se usa para reorganizar un índice es ALTER INDEX con las opciones DEALLOCATE UNUSED para liberar el espacio ocupado, COALESCE para fusionar bloques adyacentes y disminuir el espacio ocupado por esos nodos, SHRINK SPACE para compactar los registros del índice y REBUILD para reorganizar su almacenamiento en su totalidad.

Identificación de las tablas a optimizar

Una buena idea para identificar buenas candidatas para ajustar es consultar la vista **V\$SQLSTATS** que contiene información de uso de recursos de todas las consultas SQL que hay en la shared pool. Los datos deben ordenarse por uso:

BUFFER_GETS para consultas que usan mucha CPU

DISK_READS para consultas con alto I/O

SORTS para demasiadas ordenaciones.

Debemos comenzar ajustando las consultas con alta I/O

El texto del SQL lo encontramos en **V\$SQLTEXT**

EXPLAIN PLAN

El comando EXPLAIN PLAN revela el plan de ejecución de una sentencia determinada

EXPLAIN PLAN FOR <sentencia sql>

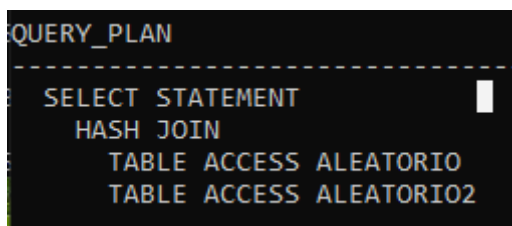
Este comando inserta datos en la tabla plan_table. Para leer esta tabla podemos usar esta query jerárquica (que podemos meter en una vista para que sea más fácil)

```
select rtrim(lpad(' ',2*level))||
       rtrim(operation)||' '||
       rtrim(options)||' '||
       object_name) query_plan
```

From plan_table connect by prior id=parent_id

Start with id=0;

A continuación una imagen que podría ser la salida de la query anterior.



```
QUERY_PLAN
-----
SELECT STATEMENT
  HASH JOIN
    TABLE ACCESS ALEATORIO
    TABLE ACCESS ALEATORIO2
```

La línea más indentada se hace primero.

El que está en el incluso nivel se hace primero lo que está más arriba.

El acceso por medio de índices tendrá un **INDEX SCAN** y un **TABLE ACCESS BY ROWID** que se hace a la vez

El **NESTED LOOP** es el acceso de los joins que usa un índice lleva una tabla (normalmente la más pequeña o la que no tiene un índice y luego usa el índice en la otra para encontrar las coincidencias).

El **HASH JOIN** es para tablas grandes, carga la pequeña en memoria y hace una hashtable por la condición de xunción

El **MERGE JOIN** no usa índices y ordena los dos conjuntos de valores para encontrar luego las coincidencias

El **AND EQUALS** usa dos índices por separado y luego mezcla los resultados

TABLE FULL ACCESS hace un full scan. Lee todos los valores de la tabla

INDEX RANGE SCAN usa un índice que devuelve más de un valor (porque no es un índice único)

INDEX UNIQUE SCAN usa un índice único

AUTOTRACE

Podemos poner el sqlplus en modo autotrace para lo cual ejecutamos el script plustrace.sql que se encuentra en el ORACLE_HOME/sqlplus/admin como SYS y luego concedemos el rol plustrace al usuario.

Luego si ejecutamos:

SET AUTOTRACE TRACEONLY EXPLAIN STATISTICS

Este comando inserta datos en la tabla plan_table y luego devuelve el plan

Debemos fijarnos en los consistent gets (que son los I/O lógicos) para optimizar las consultas.

Tunning de SQL

1. Evitar el acceso 'accidental' FULL SCAN

A veces tenemos un índice creado pero el planificador no puede tomar ventaja de él por culpa de la construcción del SQL. Algunos tipos de queries que no usan los índices son:

- a) Queries con != o NOT EQUALS
- b) Búsquedas de valores nulos
- c) Aplicamos una función a la columna del índice tal como upper(nombre)=upper('pepe')
- d) Aplicamos wildcards al principio en un like como por ejemplo like '%epe'

La primera a veces puede ser rehecha usando IN, OR, > etc. A segunda puede ser rehecha prescindiendo de la lógica trivaluada y poniendo un valor por defecto.

Un índice en un campo que contiene nulos será efectivo solo de cumplirse qué:

- pocas tuplas tienen valores
- no queremos buscar los valores nulos, sino los no nulos

2. Índices concatenados

Los índices concatenados se usarán mejor si:

- a) Contiene todos los atributos de la cláusula where
- b) el orden de las columnas de un índice concatenado permite un rango amplio de queries
- c) la columna más selectiva aparece antes (esto se pega con el apartado anterior, hay que llegar a un compromiso)
- d) siempre que sea posible, el índice puede contener los atributos del select para que así todo se satisfaga con el índice y no tengamos que acceder a los valores de la tabla.

3. SUBQUERIES

Siempre que es posible, Oracle transforma las subqueries en Joins (sobre todo si hay una clave o índice de por medio) es buena idea transformar las subqueries en joins siempre que sea posible o en caso en que no se pueda, ajustar la subquery y la otra por separado. En las subqueries correladas, hay que tener en cuenta que la subquery se ejecutará un número determinado de veces y por tanto hay que optimizarla de tal forma que se resuelva con un índice y a ser posible que se resuelva ya dentro del índice usando uno concatenado.

4. AGREGACIÓN Y ORDENACIÓN

En los ORDER BY se debe si la query tarda mucho, crear un índice en los campos del ORDER BY junto con los del select para que la sentencia se ejecute en el propio índice sin tener que reservar memoria para las ordenaciones.

Si contamos las filas de una tabla es mejor contar un índice único y sino (*). Es decir mejor count(dni) y si no count(*)

Se usará un índice en una query de agregación siempre que el índice contenga todos los campos del group by y del select

5. Consideraciones sobre el modelo

Crear claves primarias artificiales con atributos numéricos y secuencias en lugar de tener claves naturales:

- los índices son más pequeños
- las artificiales no se actualizan y las naturales hay que actualizarlas en cascada

Usar VARCHAR2 en lugar de CHAR porque ahorra espacio

Ten cuidado con los valores nulos ya que no figuran en los índices. Muchas veces es preferible un campo no nulo con un valor por defecto como "desconocido" o similar para poder utilizar los índices.

Replicar atributos para evitar joins excesivos. Esto puede ser útil en tablas estáticas. La consistencia de los atributos copiados debe ser custodiada por triggers.

Queries que hacen operaciones de agregación pueden ser muy costosas cuando hay muchas tuplas. Mejor realizar un campo calculado que se actualice con un trigger en tiempo real o con un proceso de batch.

Crear campos con las funciones insertadas (tipo upper...) controlados por triggers es mejor a veces que tener las funciones que no usen los índices.

Si tenemos una tabla enorme con accesos FULL SCAN inevitables podemos separarlas en varias y crear una vista para tener la tabla completa.

Si tenemos una tabla con campos largos y a los que se accede poco (tipo perfil o similar) podemos moverlos para una tabla con relación 1:1 con la original.

PRÁCTICA

INDICES

Crea una tabla con 100000 registros (create table test as select level id, 'nombre_'||level nombre from dual connect by level<=100000)

Habilita el autotrace

Mira el plan de ejecución y el coste en I/O de la consulta select * from test where id=1000;

Crea un índice y vuelve a mirar el plan de ejecución.

Analiza la tabla y vuelve a mirar el plan de ejecución.

Crea dos tablas

tabla1(id number, nombre varchar2(128), sexo char(1))

tabla2(id number,nombre varchar2(128), t1 number references tabla1(id));

Inserta en ellas 1 millón de valores aleatorios

Mira el plan de ejecución con explain plan de la query select a.nombre,b.nombre from tabla1 a join tabla2 b on a.id=b.t1;

Crea la pk en tabla1 y mira el plan de ejecución

Crea la pk en tabla2 y mira el plan de ejecución

Mira el plan de ejecución de la query select * from tabla1 where sexo='H';

Que tipo de índice deberías hacer? Hazlo y vuelve a mirar el plan de ejecución.